

Git cherry pick

`git cherry-pick` is a powerful command that enables arbitrary Git commits to be picked by reference and appended to the current working HEAD. Cherry picking is the act of picking a commit from a branch and applying it to another. `git cherry-pick` can be useful for undoing changes. For example, say a commit is accidentally made to the wrong branch. You can switch to the correct branch and cherry-pick the commit to where it should belong.

When to use git cherry pick

`git cherry-pick` is a useful tool but not always a best practice. Cherry picking can cause duplicate commits and many scenarios where cherry picking would work, traditional merges are preferred instead. With that said `git cherry-pick` is a handy tool for a few scenarios...

Team collaboration

Often times a team will find individual members working in or around the same code. Maybe a new product feature has a backend and frontend component. There may be some shared code between two product sectors. Maybe the backend developer creates a data structure that the frontend will also need to utilize. The frontend developer could use `git cherry-pick` to pick the commit in which this hypothetical data structure was created. This pick would enable the frontend developer to continue progress on their side of the project.

Bug hotfixes

When a bug is discovered it is important to deliver a fix to end users as quickly as possible. For an example scenario, say a developer has started work on a new feature. During that new feature development they identify a pre-existing bug. The developer creates an explicit commit patching this bug. This new patch commit can be cherry-picked directly to the `main` branch to fix the bug before it effects more users.

Undoing changes and restoring lost commits

Sometimes a `feature` branch may go stale and not get merged into `main`. Sometimes a pull request might get closed without merging. Git never loses those commits and through commands like [git log](#) and [git reflog](#) they can be found and cherry picked back to life.

How to use git cherry pick

To demonstrate how to use `git cherry-pick` let us assume we have a repository with the following branch state:

```
1  a - b - c - d   Main
2          \
3          e - f - g Feature
```

`git cherry-pick` usage is straight forward and can be executed like:

```
1  git cherry-pick commitSha
```

In this example `commit Sha` is a commit reference. You can find a commit reference by using `git log`. In this example we have constructed lets say we wanted to use commit `f` in `main`. First we ensure that we are working on the `main` branch.

```
1  git checkout main
```

Then we execute the cherry-pick with the following command:

```
1  git cherry-pick f
```

Once executed our Git history will look like:

```
1  a - b - c - d - f   Main
2          \
3          e - f - g Feature
```

The `f` commit has been successfully picked into the main branch

Examples of git cherry pick

`git cherry pick` can also be passed some execution options.

```
1  -edit
```

Passing the `-edit` option will cause git to prompt for a commit message before applying the cherry-pick operation

```
1  --no-commit
```

The `--no-commit` option will execute the cherry pick but instead of making a new commit it will move the contents of the target commit into the working directory of the current branch.

```
1  --signoff
```

The `--signoff` option will add a 'signoff' signature line to the end of the cherry-pick commit message

In addition to these helpful options `git cherry-pick` also accepts a variety of merge strategy options. Learn more about these options at the [git merge strategies](#) documentation.

Additionally, git cherry-pick also accepts option input for merge conflict resolution, this includes options: `--abort` `--continue` and `--quit` this options are covered more in depth with regards to [git merge](#) and [git rebase](#).

Summary

Cherry picking is a powerful and convenient command that is incredibly useful in a few scenarios. Cherry picking should not be misused in place of [git merge](#) or [git rebase](#). The [git log](#) command is required to help find commits to cherry pick.